

SYDE671 – Assignment 1

[Credit: taken from Berkeley CS180 intro to computer vision course, project 1]

PART ONE. (fun exercise)

1. Becoming Friends with your Camera

In this class, we want you to *become friends with every pixel*. The first step toward this goal is to become friends with your camera—e.g., the one in your smartphone. In particular, the goal of this project is to get you some intuitive understanding of the somewhat subtle relationship between perspective, focal length/zoom, and the center of projection. Don't worry if these terms aren't familiar to you yet. For now, your task is to just take some photos with your camera, as described below:

Part 1: Selfie: The Wrong Way vs. The Right Way

Take a picture of your friend (or yourself) from close up. You get a typical distorted selfie image. Now step back several feet from your subject, zoom in, and take a second picture. Try to get the face in the second photo to be the same size as in the first photo. If you've done things right, the second portrait should look much better than the first one. Think about why this is.



85mm @ 200cm

35mm @ 85cm

16mm @ 40cm

12mm @ 30cm

8mm @ 20cm

Part 2: Architectural Perspective Compression

Let's repeat the same procedure in reverse, for an urban scene. Pick a nice view down a long street (or a walking path on campus), zoom in, and take a photo. Now, walk down the street in the direction of your first shot, and take a second photo without zoom, such that the scene in the two photos appears approximately the same size. The first picture should look flattened, or compressed, compared to the second. **Again, think about why.**

Part 3: The Dolly Zoom

In this part, we will be duplicating a classic film effect called the [dolly zoom](#). It is a camera movement also known as the "Vertigo shot", after a famous scene in Hitchcock's Vertigo. Here are some examples of this effect in cinema:

https://youtu.be/_OO3pqJVLAc?si=L_eLJimucGnxjV2D

The idea is to simultaneously move the camera back while also zooming in. First, come up with a good setting for the shot (e.g., a scene with stuffed animals is one popular choice). Now take a few (4-8 or even more!) still photos while you move the camera back and zoom in, keeping the resulting image roughly the same size. Combine your stills into an animated GIF file—you have a dolly zoom!

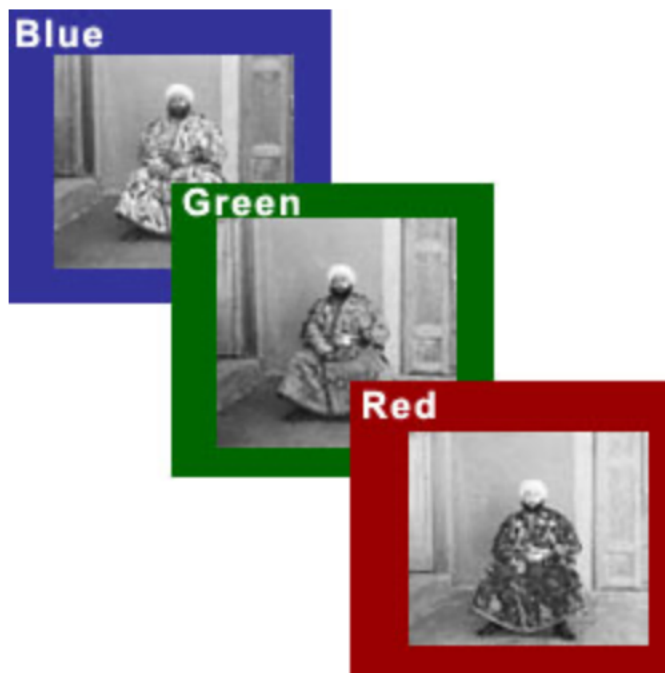
Deliverable for part 1: you must submit your project webpage. **Take beautiful photos. Make your webpage visually appealing. Aesthetics matter!**

1. **Host your webpage online.** We recommend using GitHub Pages but any hosting service is acceptable. You may organize your project in any directory structure you prefer.
2. **Submit a PDF version to learn.**

There is no grading rubric for part one however to complete the course, you will have to submit material for part one that covers all of the above.

PART TWO

Images of the Russian Empire: *Colorizing the [Prokudin-Gorskii](#) photo collection*



Background

[Sergei Mikhailovich Prokudin-Gorskii](#) (1863-1944) [Сергей Михайлович Прокудин-Горский, to his Russian friends] was a man well ahead of his time. Convinced, as early as 1907, that color photography was the wave of the future, he won Tzar's special permission to travel across the vast Russian Empire and take color photographs of everything he saw including the only color portrait of [Leo Tolstoy](#). And he really photographed everything: people, buildings, landscapes, railroads, bridges... thousands of color pictures! His idea was simple: record three exposures of every scene onto a glass plate using a red, a green, and a blue filter. Never mind that there was no way to print color photographs until much later -- he envisioned special projectors to be installed in "multimedia" classrooms all across Russia where the children would be able to learn about their vast country. Alas, his plans never materialized: he left Russia in 1918, right after the revolution, never to return again. Luckily, his RGB glass plate negatives, capturing the last years of the Russian Empire, survived and were purchased in 1948 by the Library of Congress. The LoC has recently digitized the negatives and made them available on-line.

Overview

The goal of this assignment is to take the digitized Prokudin-Gorskii glass plate images and, using image processing techniques, automatically produce a color image with as few visual artifacts as possible. In order to do this, you will need to extract the three color channel images, place them on top of each other, and align them so that they form a single RGB

color image. [This](#) is a cool explanation on how the Library of Congress composed their color images.

We will assume that a simple x,y translation model is sufficient for proper alignment. However, the full-size glass plate images (i.e. .tif files) are very large, so your alignment procedure will need to be relatively fast and efficient. When you begin your naive implementation, you should start with the smaller files `monastery.jpg` and `cathedral.jpg` provided, or by downsizing the larger files. Your submission should be run on the full-size images.

<https://www.loc.gov/pictures/item/2018678943/>

Sergey Prokudin-Gorsky created early color photographs by capturing three separate black-and-white exposures of the same scene on a single vertical glass plate using red, green, and blue color filters. They are organized BGR from top to bottom. Use the stacked images to re-create the colour image.

Details



A few of the digitized glass plate images are available from the collection. You will have to create your own low-res versions. Pick 3 plate sets for experiments. Show intermediate results. Your program will take a glass plate image as input and produce a single color image as output. The program should divide the image into three equal parts and align the second and the third parts (e.x. G and R) to the first (B). For each image, you will need to print the (x,y) displacement vector that was used to align the parts.

The easiest way to align the parts is to exhaustively search over a window of possible displacements (say $[-15, 15]$ pixels), score each one using some image matching metric, and take the displacement with the best score. There are several possible metrics that one could use to score how well the images match. The simplest one is just the L2 norm also known as the **Euclidean Distance** which is simply $\sqrt{\sum(\sum((\text{image1} - \text{image2})^2))}$ where the sum is taken over the pixel values. Another is **Normalized Cross-Correlation (NCC)**, which is simply a dot product between two normalized vectors: $(\text{image1} / \|\text{image1}\| \text{ and } \text{image2} / \|\text{image2}\|)$.

Exhaustive search will become prohibitively expensive if the pixel displacement is too large (which will be the case for high-resolution glass plate scans). In this case, you will need to implement a faster search procedure such as an image pyramid. An image pyramid represents the image at multiple scales (usually scaled by a factor of 2) and the processing is done sequentially starting from the coarsest scale (smallest image) and going down the pyramid, updating your estimate as you go. It is very easy to implement by adding recursive calls to your original single-scale implementation. You should implement the pyramid functionality yourself using appropriate downsampling techniques.



Your job will be to implement an algorithm that, given a 3-channel image, produces a color image as output. Implement a simple single-scale version first, using for loops, searching over a user-specified window of displacements. You should pick one of the

smaller .jpg images (low-res) to test this version of the code. Next, add a coarse-to-fine pyramid speedup to handle large images.

Note that in the case like the Emir of Bukhara (show above), the images to be matched do not actually have the same brightness values (they are different color channels), so you might have to use a cleverer metric, or different features than the raw pixels. This image is a great candidate for a *Bells & Whistles* extension if you want to explore more advanced alignment strategies or heuristics.

However, for grading, we allow up to **one** image (out of the original 5, excluding your own) to be misaligned in your final results; aim to get the rest properly aligned.

Bells & Whistles

Although the color images resulting from this automatic procedure will often look strikingly real, they are still a far cry from the manually restored versions available on the LoC website and from other professional photographers. Of course, each such photograph takes days of painstaking Photoshop work, adjusting the color levels, removing the blemishes, adding contrast, etc. Can we make some of these adjustments automatically, without the human in the loop?

You can use any libraries to solve bells and whistles as long as you can explain what it is doing and why you used it.

- **Automatic cropping.** Remove white, black or other color borders. Don't just crop a predefined margin off of each side -- actually try to detect the borders or the edge between the border and the image.
- **Automatic contrasting.** It is usually safe to rescale image intensities such that the darkest pixel is zero (on its darkest color channel) and the brightest pixel is 1 (on its brightest color channel). More drastic or non-linear mappings may improve perceived image quality.
- **Automatic white balance.** This involves two problems -- 1) estimating the illuminant and 2) manipulating the colors to counteract the illuminant and simulate a neutral illuminant. Step 1 is difficult in general, while step 2 is simple (see the Wikipedia page on [Color Balance](#) and section 2.3.2 in the [Szeliski book](#)). There exist some simple algorithms for step 1, which don't necessarily work well -- assume that the average color or the brightest color is the illuminant and shift those to gray or white.
- **Better color mapping.** There is no reason to assume (as we have) that the red, green, and blue lenses used by Produkin-Gorskii correspond directly to the R, G,

and B channels in RGB color space. Try to find a mapping that produces more realistic colors (and perhaps makes the automatic white balancing less necessary).

- **Better features.** Instead of aligning based on RGB similarity, try using gradients or edges.

Feel free to come up with your own approaches. There is no right answer here -- just try out things and see what works. For example, the borders of the photograph will have strange colors since the three channels won't exactly align. See if you can devise an automatic way of cropping the border to get rid of the bad stuff. One possible idea is that the information in the good parts of the image generally agrees across the color channels, whereas at borders it does not.

- **Better transformations.** Instead of searching for the best x and y translation, additionally search over small scale changes and rotations. Adding two more dimensions to your search will slow things down, but the same course to fine progression should help alleviate this.
- **Aligning and processing data from other sources.** In many domains, such as astronomy, image data is still captured one channel at a time. Often the channels don't correspond to visible light, but NASA artists stack these channels together to create false color images. For example, this [tutorial](#) on how to process Hubble Space Telescope imagery yourself. Also, consider images like [this one of a coronal mass ejection](#) built by combining [ultraviolet images](#) from the Solar Dynamics Observatory. To truly show that your algorithm works, you should demonstrate a non-trivial alignment and color correction that your algorithm found.

Deliverables

For this assignment, you must submit both your code and a project webpage in learn as a zip file for the code and a webpage link as well as a pdf of the webpage.

The project webpage is your presentation of your work. Imagine that you are writing a blog post about your project for your friends. A good blog post is easy to read and follow, well organized, and visually appealing.

When you introduce new concepts or tricks that improve your results, explain them along the way and show the improved results of your algorithm on example images.

Below are the specific deliverables to keep in mind when writing your project webpage.

- The results of a single-scale alignment (using NCC/L2 norm metrics) on the low-resolution images (JPEG files).

- The results of a multi-scale pyramid alignment (using NCC/L2 norm metrics) on **all** of the example images uploaded to the learn site.. List the offsets you computed.
- The results of your algorithm (using NCC/L2 norm metrics) on a few examples of your choosing, downloaded from the [Prokudin-Gorskii collection](#).
- If your algorithm failed to align any image, provide a brief explanation of why.
- Describe any bells and whistles you implemented. For maximum credit, show before-and-after images.

Final Advice

- You'll might build image pyramids again in the next assignment—write clean, reusable code.
- Implement almost everything from scratch. It's fine to use functions for reading, writing, resizing, shifting, and displaying images (e.g., `imread`, `imresize`, `circshift`), but don't use high-level functions for Laplacian/Gaussian pyramids, automatic alignment, etc. If in doubt, ask on Ed.
- Aim for under 1 minute per image. If it takes hours, optimize.
- Vectorize/parallelize and avoid many for-loops. See Python performance tips and NumPy broadcasting: [Python · NumPy Broadcasting](#).
- Use a fixed set of parameters; don't over-tune per image. One failure is okay with simple metrics.
- Convert images to floats and the same scale (e.g., `im2double/im2uint8`). JPGs are `uint8`; TIFFs may be `uint16`.
- Shift arrays with `np.roll`.
- Ignore borders when scoring; compute metrics on interior pixels.
- Save outputs as JPG to reduce disk usage.

Images to Use

From: <https://www.loc.gov/collections/prokudin-gorskii/?sp=3>

Use all the images in the data directory in the learn site.

Grading Rubric

This assignment will be graded out of **100** points, as follows:

Single-scale alignment (60 points total)

For results:

- **+50%:** No alignment defects on “cathedral,” “monastery,” “tobolsk.” (Assume single-scale is satisfied if pyramids work.)
- **+30%:** Some defects on “cathedral,” “monastery,” “tobolsk.”
- **0%:** No effort on alignment.

For presentation:

- **+50%:** Thorough explanation / approach / good presentation.
- **+30%:** Explanation present, could go further in depth.
- **+20%:** Minimal explanation on webpage.
- **0%:** No section / no explanation on webpage.

Multi-scale pyramid alignment (with NCC / L2) (40 points total)

For results:

- **+50%:** Alignment defects on ≤ 1 / 20 images.
- **+40%:** Alignment defects on ≤ 3 / 20 images, or missing.
- **+30%:** Alignment defects on ≤ 6 / 20 images, or missing.
- **+20%:** Alignment defects on > 6 / 20 images, but effort shown.
- **0%:** No effort on alignment.

For presentation:

- **+50%:** Thorough explanation, walking through each step carefully (e.g., NCC / L2).
- **+40%:** Explains motivation / approach / good presentation.
- **+30%:** Explanation present, could go further in depth.
- **+20%:** Minimal explanation on webpage.
- **0%:** No explanation on webpage.

Common Questions

Q: What's considered a good alignment vs. a bad alignment?

Since one failure is allowed while still receiving full credit for alignment, aim for strong results on most images (with a few failures) rather than acceptable-but-mediocre results on all images.

Okay



Not Okay



Q: What if I used a better distance function beyond L2 and NCC to get better alignments?

That's great and encouraged. However, to receive full credit you must still document results using the basic distance functions (L2/NCC). If you skip this, your presentation score will be penalized.

Using GITHUB

TL;DR: If you make a repo called <your_github_username>.github.io, GitHub will host your site for free! Then it's as simple as adding an index.html file to the repo and building on top of it.

1. **Log in to GitHub:**
2. **Create a New Repository:**

- [Go here](#), or to your GitHub dashboard and click the green "New" button to create a new repository.
- Name your repository <your_github_username>.github.io (replace <your_github_username> with your actual GitHub username).
- Set the repository to be **public** -- course staff will need access when grading.
- Start with a README file so the repo isn't empty.
- Click the "Create repository" button.

Step 2: Add an index.html File

1. **Navigate to Your New Repository:** After creating the repository, you'll be redirected to the repository page.
2. **Create an index.html File:**
 - Click the "Add file" button, then select "Create new file."
 - Name the file index.html.
 - In the file editor, you can start with something simple like:
 - `<!DOCTYPE html>`
 - `<html>`
 - `<head>`
 - `<title>My Portfolio</title>`
 - `<style>`
 - `body{`
 - `font-family: Arial, sans-serif;`
 - `margin: 20px;`
 - `}`
 - `</style>`
 - `</head>`

- `<body>`
- `<h1>Welcome to My SYDE671 Portfolio!</h1>`
- `<a href="/1/index.html">Assignment 1</a>`
- `</body>`
- `</html>`
- Scroll down and commit your new file by clicking "Commit new file."

Step 3: Access Your Website

1. **Wait a Few Minutes:** GitHub will automatically deploy your site. They say this may take up to 10 minutes. This is your time to brew some tea.
2. **Visit Your New Site:**
 - Go to `https://<your_github_username>.github.io` in your web browser.
 - You should see the content of your `index.html` file displayed as your website.

Step 4: Assignment Pages

Right now, if you copied the example HTML above, you'll notice the 'Project 1' URL returns 404 as it links to a page that doesn't (yet) exist.

For each project, you should create a separate directory/subpage that you'll link to from the main page. An example directory structure is below:

`<your_github_username>.github.io/`

├─index.html (*main page*)

├─1/

| └─index.html

| └─media/

| ├─img1.jpg

| └─:

| └─imgN.jpg

└─2/

└─:

Each time you commit changes, GitHub Pages will update your site in up to 10 minutes. For this reason, we recommend cloning and editing locally so you can witness your modifications real-time and then push once satisfied.

Step 5: Customization

- You can customize the look of your site by modifying the CSS `<style>` tag in the `<head>` of your `index.html` file.
- Alternatively, you could create a separate `style.css` file; if you wanted to re-use this styling across projects pages, you'd place it in the root directory and import it *absolutely* with `<link rel="stylesheet" href="style.css">` in the `<head>` tag.
- Otherwise, you could place the style file in the project directory and reference the *relative* path as `<link rel="stylesheet" href="./style.css">`.
 - This is similar to images: ``

We aren't going to be sticklers about the design of your site, but it should be easy to navigate and understand. You shouldn't bog the site with deliverable images that take up the whole page, for example.